

Explanation of Implementation

Image Segmentation

All images are loaded and saved with OpenCV and reshaped into (number of pixels, number of features) in order to convert it to a datatype that KMeans accepts.

a. *Grey scale K-Means Segmentation*

The image is converted from color into grey scale, which reduces the number of features each pixel has to a single brightness value (instead of the usual RGB values). K-Means clusters the pixels into three intensity groups because k was chosen to be 3. Then, the cluster labels were scaled back into the 0-255 pixel range so that the segmentation could be visualized.

b. *Color-Based K-Means Segmentation*

Each of the pixels has three features (in OpenCV they are loaded as BGR), and KMeans is then applied to separate the pixels into eight clusters ($k = 8$). The learned cluster centers are then used to reconstruct the image because each pixel is then colored by the RGB value of its cluster center in order to produce a color segmented version of the original image. Another version of colorization was also implemented where, instead of directly using the RGB value of the cluster centers, randomly generated, unique colors were used for visualization.

c. *Color + Normal-Based Segmentation*

This implementation incorporates surface geometry into color segmentation. It extracts structural information from the normal map and computes surface steepness through projects of z onto the x - y plane. The normal discontinuity function calculates the gradients to detect changes in surfaces, allowing for information regarding geometric boundaries to be extracted. The discontinuity features are then normalized, their influence is weighted using the hyperparameter normal weight, and then these features are concatenated with the color features. Finally, KMeans is run (with $k = 8$) and this combined feature vector allows for segmentation that is more representation of the true object boundaries and colors. Like the color-only segmentation, two visualizations were produced: colored based on cluster center values and colored with random, unique values.

Tic Tac Toe

The value of k for k -fold cross validation was 10.

- *Classifiers*

Three classifiers were evaluated: Linear SVM, KNN, and MLP from the scikit learn library. The SVM uses a balanced class weight to account for uneven distributions of wins/losses, and the MLP uses the StandardScaler because neural networks are sensitive to the magnitude of features. The data was first shuffled and then evaluated using k -fold cross-validation. After computing predictions across all folds, the confusion matrix was built and then row-normalized. The accuracy of each model is calculated by comparing the array of all true elements with the array of model specific predictions.

- *Regressors*

Three regressors were evaluated: Linear regression, KNN, and MLP. After data was shuffled, symmetry-based data augmentation was performed. Each board state was rotated and flipped in 8 symmetric ways in order to increase the dataset size, thus increasing the models' resulting accuracy. Linear regression was implemented manually using the normal equation, and all 9 outputs were trained simultaneously. Argmax one-hot encoding was used for linear regression to convert predictions into legal move vectors.

Rounding was used to achieve this for KNN and MLP. Exact-match requirements were used for accuracy, so all 9 outputs had to match perfectly. This program was run twice, once where all of the data was used, and another time where 1/10 of the data was used in order to observe how these regressors respond to reduced training dataset size.

- *Human Gameplay*

The playable tic tac toe game uses regressors trained on the intermediate dataset for the entire game. At the start of the game, the user can choose to play against a computer that is using the linear regression, KNN, or MLP models. In order to calculate the computer's next move, the model outputs a score for each square on the current board and sorts them in descending order in order to selected the highest-scoring possible move. The main game loop includes win detection, validation of user input, draw detection, and the option to start up a new game after the current one ends. c

Image Segmentation Results

Segmentation on color only:

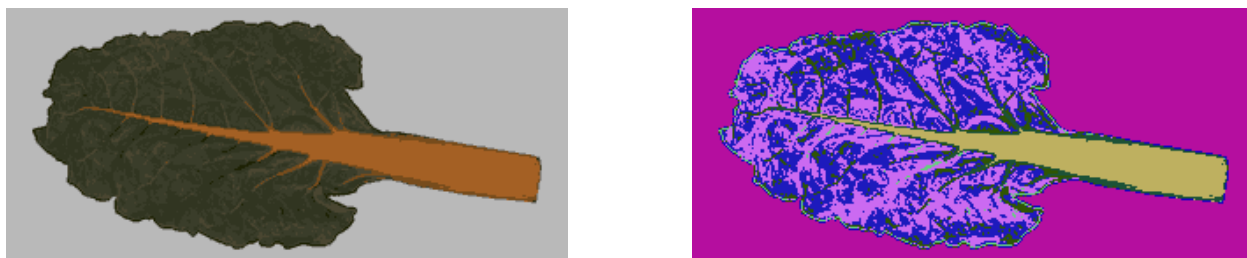


Figure 1: Segmentation colored by cluster centers (left) and segmentation colored by randomly selected unique colors (right)

Segmentation on color and normals:

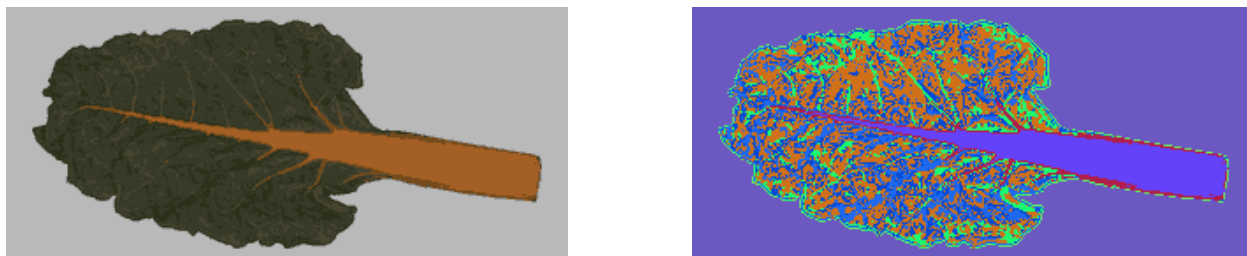


Figure 2: Segmentation colored by cluster centers (left) and segmentation colored by randomly selected unique colors (right)

Explanation of modified algorithm:

The modified algorithm adds information from the normal map to indicate changes in surface shape. Each pixel now has four features instead of three: [R,G,B, normal discontinuity]. To extract the surface geometry, the program converts the normal map into unit normals in three dimensions. The surface steepness is then computed and stored in the scalar s .

Computing surface discontinuity magnitude, where a high value indicates a sharp boundary and a low value indicates a smooth surface, allows for geometry-based silhouette detection. The combined feature vector

concatenates the color features with this newly calculated surface discontinuity feature, and the normal weight controls how strongly the geometry influences clustering compared to color. Therefore, clustering now depends on both pixels' similarity in color and their similarity in shape.

This modification improves image segmentation because small details, such as the veins in the leaves and 3D shape of the chard leaves, were not as defined during segmentation with color alone due to lack of detail across object boundaries. By combining color and shape features, objects with the same or similar colors but different shapes are separated, and boundaries between features in the image become sharper.

Evaluation of Tic Tac Toe Boards

Intermediate Dataset Performance:

SVM:

Accuracy: 0.1698977255380858

Normalized Confusion Matrix:

```
[[0.40664962 0.13107417 0.06905371 0.17519182 0.08120205 0.04219949
 0.04411765 0.03132992 0.01918159]
 [0.36568047 0.09467456 0.10295858 0.15857988 0.09704142 0.04615385
 0.04023669 0.04733728 0.04733728]
 [0.30524152 0.17060637 0.14182939 0.12641316 0.05755396 0.09969168
 0.00102775 0.02466598 0.0729702 ]
 [0.26120858 0.14035088 0.0994152 0.208577 0.07212476 0.12475634
 0.07212476 0.00974659 0.01169591]
 [0.30497132 0.20172084 0.06405354 0.18546845 0.0583174 0.09847036
 0.02294455 0.04971319 0.01434034]
 [0.2776204 0.08498584 0.17847025 0.16713881 0.04815864 0.0878187
 0.04815864 0. 0.10764873]
 [0.29595588 0.09191176 0.10661765 0.19301471 0.09375 0.06433824
 0.06617647 0.05147059 0.03676471]
 [0.33070866 0.22047244 0.13385827 0.01181102 0.09055118 0.
 0.13779528 0.00787402 0.06692913]
 [0.25708061 0.1416122 0.1416122 0.18518519 0.01525054 0.06100218
 0.07189542 0.07843137 0.04793028]]
```

(a) Support Vector Machine

KNN:

Accuracy: 0.7621737139368036

Normalized Confusion Matrix:

```
[[0.86828645 0.0140665 0.03644501 0.01150895 0.02237852 0.00959079
 0.02173913 0.00575448 0.01023018]
 [0.06035503 0.73017751 0.04378698 0.04378698 0.05680473 0.00710059
 0.02366864 0.00710059 0.02721893]
 [0.08838643 0.05138746 0.76978417 0.01849949 0.03186023 0.00924974
 0.01336074 0.0020555 0.01541624]
 [0.09356725 0.06432749 0.03898635 0.65497076 0.05263158 0.01754386
 0.04483431 0.00974659 0.02339181]
 [0.07552581 0.0458891 0.05066922 0.02007648 0.791587 0.00286807
 0.00478011 0.00382409 0.00478011]
 [0.05665722 0.07932011 0.02832861 0.02832861 0.03116147 0.73371105
 0.01983003 0.00849858 0.01416431]
 [0.09926471 0.05514706 0.02757353 0.04963235 0.02022059 0.02757353
 0.70772059 0.00735294 0.00551471]
 [0.05511811 0.08661417 0.03149606 0.04330709 0.03937008 0.03543307
 0.04724409 0.64173228 0.01968504]
 [0.11764706 0.05664488 0.04793028 0.03703704 0.03921569 0.03267974
 0.0087146 0.01089325 0.64923747]]
```

(b) K-Nearest Neighbors

MLP:

Accuracy: 0.7933139978629217

Normalized Confusion Matrix:

```
[[0.89769821 0.00959079 0.02237852 0.00959079 0.03516624 0.00639386
 0.00703325 0.00511509 0.00703325]
 [0.04142012 0.78106509 0.0556213 0.0260355 0.04733728 0.01065089
 0.02366864 0. 0.01420118]
 [0.0606372 0.04522097 0.7995889 0.02466598 0.04316547 0.0061665
 0.00719424 0.00719424 0.0061665 ]
 [0.06042885 0.05068226 0.02534113 0.6920078 0.06042885 0.00974659
 0.04483431 0.02534113 0.03118908]
 [0.0583174 0.02294455 0.02007648 0.00764818 0.85755258 0.00764818
 0.01338432 0.00382409 0.00860421]
 [0.08215297 0.09065156 0.05665722 0.04532578 0.01983003 0.57507082
 0.06232295 0.03399433 0.03399433]
 [0.08455882 0.04227941 0.03492647 0.04963235 0.01654412 0.03125
 0.72977941 0.00367647 0.00735294]
 [0.11417323 0.09448819 0.07874016 0.04724409 0.03149606 0.04330709
 0.01968504 0.51181102 0.05905512]
 [0.06971678 0.03703704 0.01960784 0.0087146 0.03050109 0.0130719
 0.00217865 0.00653595 0.81263617]]
```

(c) Multilayer Perceptron

Figure 3: Classifier performance on the intermediate board dataset.

```
Final Dataset Performance:
SVM:
Accuracy: 0.9832985386221295
Normalized Confusion Matrix:
[[0.95180723 0.04819277]
 [0.         1.         ]]

KNN:
Accuracy: 1.0
Normalized Confusion Matrix:
[[1. 0.]
 [0. 1.]]

MLP:
Accuracy: 0.8674321503131524
Normalized Confusion Matrix:
[[0.70180723 0.29819277]
 [0.04472843 0.95527157]]
```

Figure 4: Performance of classifiers on the final board dataset.

```
Intermediate Dataset (Multi-Label) Regressor Performance:
Linear Regression Accuracy:
0.12181346359334452
KNN Regression Accuracy:
0.9963555182414898
MLP Regression Accuracy:
0.7619447412608762
```

Figure 5: Exact-match accuracy comparison of Linear Regression, KNN, and MLP regressors on the intermediate multi-label dataset.

Classifier Performance Analysis

K-Nearest Neighbors performed the best on the final board dataset, with a perfect accuracy. This is because the data was low dimensional (with 9 features) and had highly structured, discrete values. Additionally, many board configurations are very similar. KNN performs well in these scenarios because it predicts based on nearby examples in the training data, and similar tic-tac-toe boards have similar optimal outcomes. KNN's local, memory-based predictions are well suited for the small dataset.

On the intermediate dataset, MLP achieved the highest accuracy, while KNN was very close behind. SVM likely performed poorly because this is not a linear problem, and the linear decision boundary cannot predict the nonlinear interactions for move selection. MLP did not drastically outperform KNN because the dataset is relatively small, so the neural network was not able to train as well because neural networks perform best with larger and more complex datasets. Once again, KNN performed well because of the small, structured, low-dimensional data it was trained on.

Regressor Performance Analysis

KNN regression's performance was significantly better. Once again, this is due to the structure of the tic-tac-toe data. Because the state space is limited and highly structured, many boards in the dataset are either identical or very similar to one another. This is especially true after the data was artificially expanded using symmetry based data augmentation. Since optimal moves are similar across similar board states, this local, memory-based strategy performs very well. With augmentation, KNN learns every board configuration under rotations and reflections, making a near-perfect generalization possible. On the other hand, linear regression

```
Linear Regression Accuracy: 0.07557251908396946
KNN Regression Accuracy: 0.6064885496183207
MLP Regression Accuracy: 0.6158396946564886
```

Figure 6: Regressor accuracies when trained on 1/10 of the original data

performs very poorly because the relationship between board positions and optimal moves is highly nonlinear, so they cannot be captured by a linear function of the inputs. MLP is limited by the dataset being relatively small and discrete, which does not allow for proper training of the neural network.

Accuracy when trained on 1/10 of the data:

When trained on 1/10th of the data, the accuracies of all regressors go down. KNN experiences the most significant drop because this regressor method relies directly on the number of memorized examples, so with fewer training samples, the likelihood that a new data point closely matches a training data point decreases. MLP also decreases, but not as significantly because MLP learns the function rather than depending on memorization of data points. However, since the dataset is so small, the neural network does not have enough data to generalize well. The linear regression model would continue to perform poorly since it still fails to represent the non-linear data properly.

Scaling with Large Datasets MLP scales better with large datasets because it utilizes a neural network to learn a function that can predict optimal moves. Training is very expensive because it requires a lot of data, but once the model is trained, predictions are fast. As the dataset grows, MLP can better approximate the underlying structures of optimal tic-tac-toe moves without relying on memorization. Therefore, the runtime and memory scale well with larger datasets.

KNN also performs very well when the dataset is large, but it does not scale as well computationally because as the dataset grows, prediction costs grow as well. This is because KNN memorizes the training examples, and since data augmentation was performed, the dataset is dense, so for most test samples, KNN can find a very similar training example to compare it to. However, due to the fact that it needs to compute differences to every training example, the runtime and memory do not scale well with large datasets.

Linear regression does not scale well with larger datasets in this context because of this model's capacity (tic-tac-toe is not a linear problem).

Tic Tac Toe Gameplay

Difficulty Against the Computer

It is very easy to beat the computer if it is using the linear regressor network, likely because that model has low accuracy.

KNN and MLP are both relatively hard to beat, and most games end up in a draw. This is likely because these two networks had significantly higher accuracy compared to the linear regression model.

Network Architecture and Algorithm

For the tic tac toe game, the network architecture is designed to predict optimal moves by reshaping the current state of a 3x3 board into a 9D vector. The MLP regressor uses `hidden_layer_sizes = (64,64)`, ReLU activation, the Adam optimizer, and input scaling using `StandardScaler()` to stabilize the training process. Linear regression computes scores using the normal equation and KNN predicts scores based on neighbors.

The move selection algorithm converts the selected network's outputs into legal moves by iterating through the sorted, most optimal moves, and picking the first unoccupied square because that is the best valid move.

Game Logic

The tic tac toe game logic handles evaluation of the board state in order to determine win/draw situations. The board is visually represented with a 9-element list, and the `WINS` list stores all of the possible winning combinations in tic tac toe. The `check_winner(board)` function checks each line in `WINS` to see if all three squares are equal, and returns 1 if the player wins and -1 if the computer wins. The `is_full(board)` function checks for draws by seeing if all squares are nonzero and that it doesn't match the winning board. `print_board(board)` formats the board for visual gameplay purposes.

Game Loop

The main game loop allows for turn-based gameplay. The player moves first, and the input is validated before it appears visually on the board. After each move, the program will check for a winner or a draw. Next, the computer chooses its move using the selected model. This loop will continue alternating turns until a win or a draw occurs.

Instructions on How to Run the Programs

To run these programs, install the dependencies in `requirements.txt`, and then run the relevant python scripts. The tic tac toe game is run in the terminal as well.

Additional Information

N/A